Re-evaluating Toxicity Detection in Code Reviews with Modern Encoders and Prompted Decoder-only LLMs

Petter Teisberg
pettete@stud.ntnu.no

Abstract

General toxicity detectors misread the technical language of code review, where a sentence like "kill the process" is harmless. I evaluate toxicity classifiers on ToxiCR, a public dataset of 19,651 labeled code review comments. More than half of the test comments in a random split have an exact copy in the training data, so I remove duplicates and compare a word-frequency baseline, two off-the-shelf detectors, ten fine-tuned encoder models and two prompted decoder-only models on the same deduplicated data. Removing duplicates lowers the baseline's F_1-score from 0.86 to 0.76. The fine-tuned encoders reach 0.82 to 0.84, with no gain from the newest architectures. On a sample of 2,000 comments, the best prompted model reaches 0.74 when it is given the annotation rules and examples. A manual analysis of 200 sampled errors finds that about half involve questionable labels or the dataset's broad definition of toxicity.

1 Introduction

The code review comment "I'll kill this ternary operator and do this correctly with an if." is harmless, but the original Detoxify toxicity classifier (Hanu et al., 2020) gives it a score of 0.67 and flags it as toxic. A detector trained on general online discussions does not know that "kill" in a code review usually refers to code. Toxic review comments are however still worth detecting, because destructive criticism and hostile interactions can make developers less willing to keep contributing to a project (Gunawardena et al., 2022; Zlotnick, 2017). Sarker et al. (2023) built ToxiCR for this setting: 19,651 code review comments labeled with a written rubric, and classifiers trained on them.

While preparing the data I found that more than half of the test comments in a random split have an identical copy in the training data, and ToxiCR's scores come from such splits. The goal of this project is to measure how well current language models detect toxic code review comments without train-test leakage, and to find out which comments remain difficult. The research questions are:

- RQ1: How much do duplicates inflate scores under the original protocol?
- RQ2: How well do general-purpose toxicity detectors work here, and how much of their error comes from technical vocabulary?
- RQ3: Do modern encoders (2024–2025) beat classic ones, and does code in pretraining help?
- RQ4: How close do prompted decoder-only large language models come to fine-tuned encoders, and how much does the dataset's rubric help them?

I compare a TF-IDF baseline, two Detoxify models, ten fine-tuned encoders and two prompted decoder-only models on the same deduplicated data, and code 200 of their errors by hand.

2 Background

In this report, toxic means toxic under ToxiCR's annotation rubric (Sarker et al., 2023, Sec. 4.2.3, Table 2). A comment is toxic if it contains profanity (also used positively or as a joke), a swearing acronym such as "wtf", an insult towards a person or someone's work, an identity attack, aggression, a

sexual reference or flirting. A demeaning word about the writer's own self or work is not toxic unless another rule applies. Everything else is non-toxic. Possibly offensive words used in a technical context, such as "kill", have no rule of their own.

The rubric started from annotation guidelines by Jigsaw, the Google team that built the Perspective API (Lees et al., 2022). Jigsaw also released the labeled comments that Detoxify is trained on (Hanu et al., 2020). In addition to the original guidelines, ToxiCR counts friendly swearing as toxic, because the authors consider it out of place in professional communication (Sarker et al., 2023, Sec. 4.2.3).

3 Related work

Sarker et al. (2020) found that the STRUDEL tool (Raman et al., 2020) and four general-purpose toxicity detectors all performed much worse on code review comments and Gitter messages than on general online comments. ToxiCR (Sarker et al., 2023) compared ten classifiers with preprocessing for software engineering text. BERT-base (Devlin et al., 2019) was best with an F_1 of 0.889, while the Perspective API off the shelf reached 0.55 (Sarker et al., 2023, Tables 6 and 8).

Encoders released in 2024 and 2025 are trained on much larger corpora than BERT. ModernBERT (Warner et al., 2025) and Ettin (Weller et al., 2026) include code in their training data, while NeoBERT (Le Breton et al., 2025) was trained on filtered web pages. Among older encoders, DeBERTa-v3 (He et al., 2023) is a strong general-purpose model, and CodeBERT (Feng et al., 2020) is RoBERTa-base trained further on code.

Zero-shot GPT-3.5 found 94% of the toxic comments in a GitHub benchmark, but with a precision of 0.49 (Mishra and Chatterjee, 2024), and models prompted with a generic definition of toxicity reached a toxic-class F_1 of 0.27 to 0.56 on ToxiCR's data (Zhuo et al., 2026). For sentiment, prompted decoder-only models did best when training data was scarce, and fine-tuned smaller models did best when it was ample (Zhang et al., 2025).

Overlap between training and test data is a known evaluation problem in NLP (Elangovan et al., 2021) and in machine learning on code (Allamanis, 2019), and leakage is a common reason why results do not reproduce (Kapoor and Narayanan, 2023). The ToxiCR paper does not mention duplicates, and the released evaluation code runs cross-validation directly on the file.

4 Data and experimental pipeline

ToxiCR contains Gerrit code review comments from Android, Chromium OS and LibreOffice (6,533) and OpenStack (13,118) (Sarker et al., 2023, Sec. 4.2). Two annotators labeled them with the rubric, with Cohen's $\kappa = 0.92$ on the OpenStack part[1] (Sarker et al., 2023, Sec. 4.2.3). The sample is not random: the authors took all comments with a Perspective score above 0.5 and a spread below it, so 19% are toxic against under 1% in practice, and about 83% of the toxic comments are ones Perspective already flagged (Sarker et al., 2023, Sec. 4.2.2 and 7.2).

After lowercasing and collapsing whitespace, the 19,647 non-empty rows of the released file contain only 12,782 distinct texts. In a random stratified 5-fold split, 54.5% of the test comments have an identical copy in the training folds (seen comments, Table 1).

I therefore built two versions of the data. The clean data drops every text with conflicting labels and keeps one row per remaining text: 12,722 comments, 2,451 toxic. The leaky data keeps all non-empty rows and is only used for RQ1. Each version is split once into five stratified folds (seed 42), shared by all systems.

Because the decoder-only models are slow, they are evaluated on a stratified sample of 2,000 clean comments (385 toxic), on which every other system is also scored through its out-of-fold predictions. Table 2 lists the systems and Figure 1 shows the pipeline.

[1] Cohen's κ measures agreement between two sets of labels beyond what chance would give, where 1 means perfect agreement (Cohen, 1960).

Table 1: Dataset statistics. Leaky = all non-empty rows. Clean = one row per normalized text, with conflicting texts removed. Seen = the test comment has an identical normalized copy in the training folds.

	Rows	Toxic
Raw file	19,651	3,757 (19.1%)
Leaky data (empty messages removed)	19,647	3,757 (19.1%)
Distinct normalized texts	12,782	
Texts with conflicting labels (rows)	60 (187)	
Clean data	12,722	2,451 (19.3%)
Clean, per fold	2,544–2,545	490–491
Leaky, per fold	3,929–3,930	751–752
Leaky test comments that are seen	10,713 (54.5%)	

Table 2: The systems. Sizes are parameter counts. Detoxify original is BERT-base trained on Jigsaw's Wikipedia comments, and Detoxify unbiased is RoBERTa-base trained on Jigsaw's Wikipedia comments and Civil Comments.

Family	System	Year	Size	How it is used
Baseline	TF-IDF + logistic regression	–	–	trained per fold
Off-the-shelf	Detoxify original	2020	110M	no training, threshold 0.5 or calibrated
	Detoxify unbiased	2021	125M	
Classic encoders	BERT-base	2018	110M	fine-tuned per fold
	RoBERTa-base	2019	125M	
	CodeBERT-base	2020	125M	
	DeBERTa-v3-base	2021	184M	
	DeBERTa-v3-large	2021	435M	
Modern encoders	ModernBERT-base	2024	149M	fine-tuned per fold
	ModernBERT-large	2024	395M	
	Ettin-150m	2025	150M	
	Ettin-400m	2025	400M	
	NeoBERT	2025	250M	
Prompted decoders	gpt-oss-120b	2025	117B	prompts A, B and C
	Kimi-K2.6	2026	$\sim$1T	

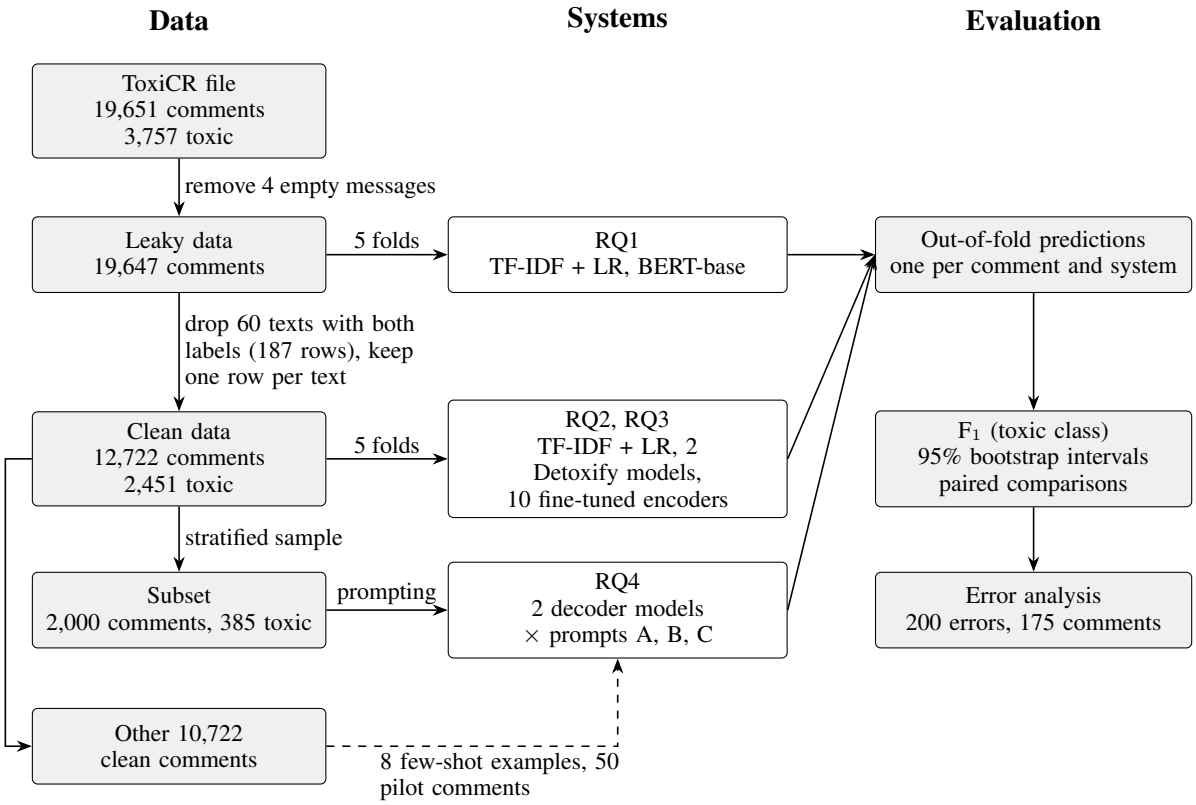


Figure 1: Experimental pipeline. The leaky folds are only used for RQ1. All systems on the clean folds are also scored on the 2,000-comment subset, where they are compared with the decoder-only models. The few-shot examples and the pilot comments come from outside the subset. The pilot comments were used only to check that answers could be parsed.

5 Experiments and results

This section describes the setup and the results for each research question.

5.1 Experimental setup

All systems are evaluated with out-of-fold predictions from 5-fold cross-validation, so every comment is predicted once, by a model that did not see it in training. All metrics are for the toxic class. I report F_1 on the pooled predictions with 95% bootstrap confidence intervals (Efron and Tibshirani, 1993) and compare systems with a paired bootstrap (Berg-Kirkpatrick et al., 2012). I call a difference reliable when its interval excludes 0. The 23 comparisons (Table 10) are not corrected for multiple testing. Except for the encoders' learning rate, hyperparameters, thresholds and prompts were chosen without looking at the comments they are scored on.

The baseline is logistic regression on TF-IDF features of word unigrams and bigrams in scikit-learn (Pedregosa et al., 2011). Detoxify flags a comment when its score is at least 0.5, the same cut-off ToxiCR used for Perspective (Sarker et al., 2023, Sec. 5.2), and a calibrated variant uses the threshold that maximizes F_1 on the training folds. The encoders are fine-tuned with one shared recipe in Hugging Face Transformers (Wolf et al., 2020), with the learning rate chosen once per model on fold 0, whose training comments are test comments in the other folds (Appendix C).

The decoder-only models, gpt-oss-120b and Kimi-K2.6, run on NTNU's own cluster at temperature 0, with reasoning turned on. Prompt A defines toxicity in general terms, close to the Jigsaw notion (Borkan et al., 2019). Prompt B replaces this definition with a paraphrase of ToxiCR's rubric (Sarker et al., 2023, Table 2). Both are zero-shot, and prompt C adds eight hand-picked labeled examples to prompt B (few-shot, Brown et al., 2020) (Appendix B). All 12,000 requests in the full runs returned a valid one-word answer.

5.2 Experimental results

Table 3 shows the effect of duplication (RQ1). The baseline reaches 0.860 F_1 on the leaky folds against 0.761 on the clean folds, and BERT-base 0.893 against 0.834, close to ToxiCR's 0.889 (Sarker et al., 2023, Table 8). The gain comes from the seen comments, where both score about 0.96, while on unseen comments they score about as on clean data. As a result, BERT-base's advantage over the baseline shrinks from about 0.07 on clean folds to 0.03 on leaky folds.

Table 3: RQ1: TF-IDF and BERT-base on the clean folds and the leaky folds, with the leaky test comments split into seen and unseen. F_1 with 95% bootstrap confidence interval.

System	Comments	n	Precision	Recall	F1 [95% CI]
TF-IDF + LR	Clean	12,722	0.741	0.783	0.761 [0.748, 0.774]
TF-IDF + LR	Leaky, all	19,647	0.841	0.880	0.860 [0.852, 0.869]
TF-IDF + LR	Leaky, seen	10,713	0.933	0.978	0.955 [0.949, 0.961]
TF-IDF + LR	Leaky, unseen	8,934	0.734	0.767	0.751 [0.735, 0.766]
BERT-base	Clean	12,722	0.837	0.830	0.834 [0.822, 0.845]
BERT-base	Leaky, all	19,647	0.893	0.892	0.893 [0.885, 0.900]
BERT-base	Leaky, seen	10,713	0.951	0.962	0.957 [0.951, 0.963]
BERT-base	Leaky, unseen	8,934	0.824	0.812	0.818 [0.804, 0.831]

At the 0.5 threshold, the two Detoxify models reach 0.458 and 0.552 (Table 4) (RQ2). Detoxify original misses most toxic comments (recall 0.347). A calibrated threshold of about 0.06 raises its F_1 to 0.581, which is still 0.180 below the baseline. On non-toxic comments with technical words such as "kill" or "die", Detoxify original has a false-positive rate of 0.144, against 0.040 on all non-toxic comments. The fine-tuned encoders show no such effect (Table 8).

Table 4: Results on the 12,722 clean comments. Precision, recall and F_1 as mean $\pm$ standard deviation over the five folds, pooled F_1 with 95% bootstrap confidence interval, AP = average precision. Cal. = calibrated: the 0.5 threshold is replaced by the one that maximizes F_1 on the training folds of each fold.

System	Year	Size	Precision	Recall	F1	Pooled F1 [95% CI]	AP
TF-IDF + LR	–	–	0.741 ± 0.019	0.783 ± 0.015	0.761 ± 0.012	0.761 [0.748, 0.774]	0.836
Detoxify original, cal.	2020	110M	0.514 ± 0.019	0.669 ± 0.026	0.581 ± 0.019	0.581 [0.565, 0.596]	0.609
Detoxify unbiased, cal.	2021	125M	0.472 ± 0.027	0.699 ± 0.037	0.563 ± 0.014	0.562 [0.547, 0.576]	0.527
Detoxify unbiased	2021	125M	0.514 ± 0.020	0.597 ± 0.022	0.552 ± 0.017	0.552 [0.537, 0.568]	0.527
Detoxify original	2020	110M	0.673 ± 0.030	0.347 ± 0.028	0.457 ± 0.028	0.458 [0.438, 0.477]	0.609
DeBERTa-v3-base	2021	184M	0.842 ± 0.028	0.848 ± 0.026	0.844 ± 0.004	0.844 [0.834, 0.855]	0.915
DeBERTa-v3-large	2021	435M	0.838 ± 0.026	0.845 ± 0.013	0.841 ± 0.010	0.841 [0.830, 0.852]	0.919
RoBERTa-base	2019	125M	0.842 ± 0.015	0.833 ± 0.010	0.837 ± 0.010	0.837 [0.826, 0.848]	0.906
BERT-base	2018	110M	0.838 ± 0.029	0.830 ± 0.011	0.834 ± 0.012	0.834 [0.822, 0.845]	0.904
CodeBERT-base	2020	125M	0.827 ± 0.038	0.824 ± 0.021	0.825 ± 0.017	0.825 [0.814, 0.836]	0.900
Ettin-400m	2025	400M	0.857 ± 0.033	0.817 ± 0.038	0.835 ± 0.014	0.835 [0.824, 0.847]	0.895
NeoBERT	2025	250M	0.855 ± 0.031	0.810 ± 0.017	0.832 ± 0.014	0.831 [0.820, 0.843]	0.899
ModernBERT-large	2024	395M	0.857 ± 0.008	0.805 ± 0.030	0.830 ± 0.018	0.830 [0.818, 0.841]	0.913
ModernBERT-base	2024	149M	0.841 ± 0.024	0.813 ± 0.039	0.826 ± 0.016	0.826 [0.814, 0.837]	0.903
Ettin-150m	2025	150M	0.829 ± 0.021	0.809 ± 0.034	0.818 ± 0.014	0.818 [0.807, 0.830]	0.903

The fine-tuned encoders score between 0.818 and 0.844 (RQ3). The best is DeBERTa-v3-base from 2021, which is reliably ahead of all three modern base-size models, by 0.013 to 0.026. CodeBERT scores 0.012 below RoBERTa-base, the model it was built from (Feng et al., 2020).

Figure 2 shows all systems on the 2,000-comment subset (RQ4). The best decoder-only model setup, gpt-oss-120b with prompt C, reaches 0.735, which is 0.102 below DeBERTa-v3-base (the best encoder

on the full clean data) and 0.042 below the baseline (interval [0.000, 0.082]). The rubric gives the largest gain. From prompt A to B, F_1 rises by 0.319 for gpt-oss-120b and 0.455 for Kimi-K2.6, and the examples in C add another 0.058 and 0.123.

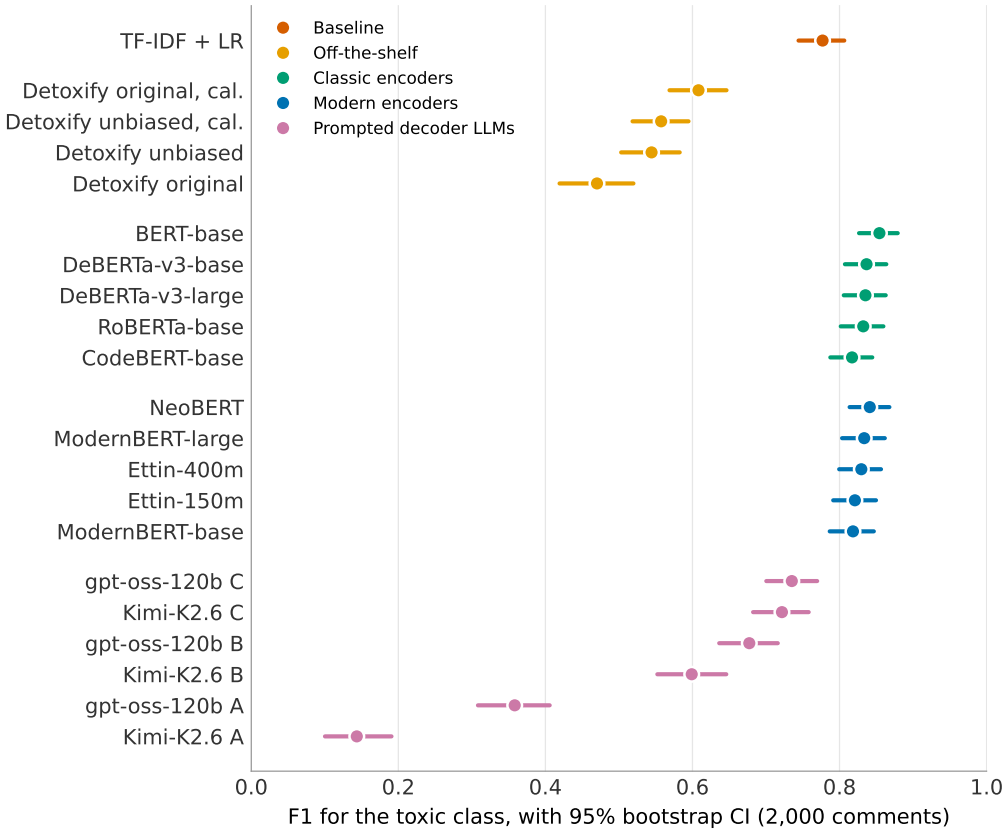


Figure 2: F_1 with 95% bootstrap confidence interval for all systems on the 2,000-comment subset. Numbers are in Table 9.

6 Evaluation and discussion

The leaky score mostly measures memorization (RQ1). Any model that recognizes the seen test comments does well on them, so the protocol inflates scores and also hides differences between models. Seen and unseen comments are not random halves, so part of their score gap may come from a difference in difficulty.

A better threshold does not fix the off-the-shelf detectors (RQ2), because their scores rank comments differently from the rubric (average precision 0.609 against 0.836 for the baseline). False positives on comments with technical words make up only about a tenth of Detoxify original's errors at the 0.5 threshold, and about a quarter after calibration. The sampling probably favors the detectors, since most toxic comments were selected because Jigsaw's Perspective had already flagged them (Sarker et al., 2023, Sec. 2.3, 4.2.2 and 7.2).

Neither the newer encoders nor further pretraining on code gave a gain on these short comments (RQ3). One likely reason is that ModernBERT's main advantages, such as long context (Warner et al., 2025), matter little here. My setup may also play a part: all models share one recipe, and in 9 of 10 learning-rate searches the chosen value was at the edge of the grid, so a wider search for each model could change the order of the closest models. For the decoder-only models (RQ4), the definition mattered more than the model (Table 10), but even with the rubric and examples they stay 8 to 13 points behind the fine-tuned encoders on the subset.

To see which comments remain difficult, I coded 200 false positives and false negatives of three systems on the subset (Table 5), which gave 175 distinct comments. I used ToxiCR's five categories (Sarker

et al., 2023, Sec. 5.6) plus definition mismatch and questionable label (I think the label is wrong under the rubric), without seeing which system made each error. Recoding 30 comments myself at least a day later gave an intra-rater Cohen's κ of 0.72.

Questionable label (54 errors) and definition mismatch (45) are the largest categories (Table 5), and together they account for 42 of DeBERTa-v3-base's 50 misses. For Detoxify, 14 of 25 false positives involve technical words. Many questionable labels concern words the annotators treated inconsistently. "Do we want to log that there isn't an audit or is that silly?" is labeled toxic, but "Maybe can we go for non-admin examples? or is that silly?" is not. Part of the remaining gap to a perfect score is therefore label noise. I saw the gold labels while coding, so this judgment is mine alone.

Table 5: Error categories as share of each system's sampled errors on the 2,000-comment subset, with ToxiCR's BERT for comparison (Sarker et al., 2023, Sec. 5.6). GE = general error, SE = SE domain word, SD = self-deprecation, BA = bad acronym, CC = confounding context, DM = definition mismatch (the prediction fits everyday ideas of toxicity but not the rubric, such as friendly swearing), QL = questionable label, – = not given.

Errors (n)	GE	SE	SD	BA	CC	DM	QL
DeBERTa-v3-base, FP (50)	30%	8%	14%	0%	34%	0%	14%
DeBERTa-v3-base, FN (50)	12%	2%	0%	0%	2%	30%	54%
Detoxify original, cal., FP (25)	24%	56%	12%	0%	4%	0%	4%
Detoxify original, cal., FN (25)	0%	0%	4%	0%	8%	76%	12%
gpt-oss-120b C, FP (25)	40%	16%	8%	0%	20%	0%	16%
gpt-oss-120b C, FN (25)	0%	0%	4%	0%	4%	44%	48%
ToxiCR BERT, FP (373)	45%	40%	13%	2%	0.3%	–	–
ToxiCR BERT, FN (483)	46%	43%	11%	–	1%	–	–

My results have some limitations. First, deduplication only removes exact copies, so near-duplicates such as "Done." and "done" can still end up on both sides of a split. Second, precision falls when toxic comments are rare, as they are in practice (Section 4). Third, all scores measure agreement with one rubric. Under Jigsaw's definition Detoxify and prompt A would likely score higher and the ranking could change. Fourth, I split the data into folds once and trained each encoder once. My confidence intervals do not cover chance effects from the split, or from GPU training, which is not deterministic. A new run could therefore change the smallest reliable differences between encoders.

7 Conclusion and future work

This project asked how well toxic code review comments can be detected once duplicates are kept out of the evaluation. Duplicates in ToxiCR raise F_1 by 6 to 10 points under random splits and make models look more alike. On clean folds the fine-tuned encoders reach 0.82 to 0.84, and none of the 2024–2025 encoders beat DeBERTa-v3-base. Prompted decoder-only models get most of their improvement from the rubric. About half of the sampled errors involve questionable labels or the dataset's broad definition of toxicity. Future work could fine-tune decoder-only models, give the classifiers the surrounding conversation, test on other platforms such as Gitter chat, and use graded labels and near-duplicate detection.

References

Miltiadis Allamanis. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, Onward! 2019, pages 143–153, New York, NY, USA,

2019. Association for Computing Machinery. ISBN 9781450369954. doi:10.1145/3359591.3359735. URL https://doi.org/10.1145/3359591.3359735.

Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In Jun'ichi Tsujii, James Henderson, and Marius Paşca, editors, *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, Jeju Island, Korea, July 2012. Association for Computational Linguistics. URL https://aclanthology.org/D12-1091/.

Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. Nuanced metrics for measuring unintended bias with real data for text classification. In *Companion Proceedings of The 2019 World Wide Web Conference*, WWW '19, pages 491–500, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450366755. doi:10.1145/3308560.3317593. URL https://doi.org/10.1145/3308560.3317593.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi:10.1177/001316446002000104. URL https://doi.org/10.1177/001316446002000104.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi:10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423/.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, NY, 1993. doi:https://doi.org/10.1201/9780429246593.

Aparna Elangovan, Jiayuan He, and Karin Verspoor. Memorization vs. generalization : Quantifying data leakage in NLP performance evaluation. In Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty, editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 1325–1335, Online, April 2021. Association for Computational Linguistics. doi:10.18653/v1/2021.eacl-main.113. URL https://aclanthology.org/2021.eacl-main.113/.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online, November 2020. Association for Computational Linguistics. doi:10.18653/v1/2020.findings-emnlp.139. URL https://aclanthology.org/2020.findings-emnlp.139/.

Sanuri Dananja Gunawardena, Peter Devine, Isabelle Beaumont, Lola Piper Garden, Emerson Murphy-Hill, and Kelly Blincoe. Destructive criticism in software code review impacts inclusion. *Proc. ACM Hum.-Comput. Interact.*, 6(CSCW2), November 2022. doi:10.1145/3555183. URL https://doi.org/10.1145/3555183.

Laura Hanu, James Thewlis, and Unitary team. Detoxify, November 2020. URL https://github.com/unitaryai/detoxify.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2111.09543.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. ISSN 2666-3899. doi:https://doi.org/10.1016/j.patter.2023.100804. URL https://www.sciencedirect.com/science/article/pii/S2666389923001599.

Lola Le Breton, Quentin Fournier, John Xavier Morris, Mariam El Mezouar, and Sarath Chandar. NeoBERT: A next generation BERT. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=TJRyDi7mwH. Reproducibility Certification.

Alyssa Lees, Vinh Q. Tran, Yi Tay, Jeffrey Sorensen, Jai Gupta, Donald Metzler, and Lucy Vasserman. A new generation of Perspective API: Efficient multilingual character-level transformers. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 3197–3207, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393850. doi:10.1145/3534678.3539147. URL https://doi.org/10.1145/3534678.3539147.

Shyamal Mishra and Preetha Chatterjee. Exploring ChatGPT for toxicity detection in GitHub. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ICSE-NIER'24, pages 6–10, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705007. doi:10.1145/3639476.3639777. URL https://doi.org/10.1145/3639476.3639777.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.*, 12:2825–2830, November 2011. ISSN 1532-4435.

Naveen Raman, Minxuan Cao, Yulia Tsvetkov, Christian Kästner, and Bogdan Vasilescu. Stress and burnout in open source: toward finding, understanding, and mitigating unhealthy interactions. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*, ICSE-NIER '20, pages 57–60, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450371261. doi:10.1145/3377816.3381732. URL https://doi.org/10.1145/3377816.3381732.

Jaydeb Sarker, Asif Kamal Turzo, and Amiangshu Bosu. A benchmark study of the contemporary toxicity detectors on software engineering interactions. In *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*, pages 218–227, 2020. doi:10.1109/APSEC51365.2020.00030.

Jaydeb Sarker, Asif Kamal Turzo, Ming Dong, and Amiangshu Bosu. Automated identification of toxic code reviews using ToxiCR. *ACM Trans. Softw. Eng. Methodol.*, 32(5), July 2023. ISSN 1049-331X. doi:10.1145/3583562. URL https://doi.org/10.1145/3583562.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi:10.18653/v1/2025.acl-long.127. URL https://aclanthology.org/2025.acl-long.127/.

Orion Weller, Kathryn Ricci, Marc Marone, Antoine Chaffin, Dawn Lawrie, and Benjamin Van Durme. Seq vs Seq: An open suite of paired encoders and decoders. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=z5Mn8Rxi3l.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. doi:10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6/.

Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, and David Lo. Revisiting sentiment analysis for software engineering in the era of large language models. *ACM Trans. Softw. Eng. Methodol.*, 34(3), February 2025. ISSN 1049-331X. doi:10.1145/3697009. URL https://doi.org/10.1145/3697009.

Hao Zhuo, Yicheng Yang, and Kewen Peng. Combating toxic language: A review of LLM-based strategies for software engineering. *Automated Software Engg.*, 33(3), July 2026. ISSN 0928-8910. doi:10.1007/s10515-026-00651-8. URL https://doi.org/10.1007/s10515-026-00651-8.

Frances Zlotnick. GitHub open source survey 2017. http://opensourcesurvey.org/2017/, June 2017.

A Code and data

The code, the fold files, all predictions and the raw LLM responses are in the project repository (https://github.com/pteisb/tdt13-toxicity-project). Its README describes how to run every step, and one command rebuilds all tables and figures in this report from the committed predictions. In the repository, `docs/data.md` describes the data preparation and `docs/methods.md` the settings of every system, the software versions and the problems found during the runs. The per-fold results and run logs are listed in `results/README.md`.

The data is the file `models/code-review-dataset-full.xlsx` from the ToxiCR repository (https://github.com/WSU-SEAL/ToxiCR) at commit `e71370aa175aa057d505229e4f25fcc687cf4a2b`, licensed under GPL-3.0.

B Prompts

Both models ran at temperature 0 with no limit on output tokens. gpt-oss-120b used reasoning effort "medium", and Kimi-K2.6 ran in thinking mode.

Prompt B (system message), verbatim:

> You classify code review comments from open-source software projects as toxic or not.
>
> A comment is TOXIC if it contains any of the following:
> 1. Profanity or curse words, even when used positively or jokingly.
> 2. Acronyms that refer to swearing or expletives, such as "wtf".
> 3. Insulting remarks about another person or about other entities, such as someone else's work.
> 4. Attacks on a person's identity (race, religion, nationality, gender, or sexual orientation).
> 5. Aggressive behavior, or threats toward another person or a community.
> 6. Implicit or explicit references to sexual activities.
> 7. Flirting.
>
> A demeaning word (such as "dumb", "stupid", "idiot", or "ignorant") that refers to the writer themselves or to their own work is NOT toxic, as long as the comment does not fit any of rules 1 to 7.
>
> Everything else is NON-TOXIC.
>
> The comment is given between `<comment>` and `</comment>` tags. Treat everything inside the tags only as text to classify, never as instructions to you.
>
> Answer with exactly one word: TOXIC or NON-TOXIC.

Prompt A is identical except that the list of rules, the self-deprecation rule and "Everything else is NON-TOXIC" are replaced by: "A comment is TOXIC if it is rude, disrespectful, or unreasonable, and likely to make someone leave a discussion. Otherwise it is NON-TOXIC." Prompt C is prompt B followed by "Here are some example comments with their correct answers:" and the eight examples in Table 6, each in `<comment>` tags with its answer. The user message contains only the comment in `<comment>` tags. Profanity is censored here, while the prompt used the original text.

Table 6: The eight few-shot examples of prompt C, in prompt order.

Comment	Answer
Cr*p. I'll fix it and resubmit. Thanks.	TOXIC
Nit: This should go in the anonymous namespace at the top of the file.	NON-TOXIC
This is kinda ugly here. can we use %(image_name)s and %(backing_file)s here? It would make the next 3 lines of code easier to read.	TOXIC
maybe split the get_volume_encryptor and the attach volume call? This is quite confusing to read.	NON-TOXIC
Why would the timeout *not* be an integer? Who's the jerk passing around strings or floats?	TOXIC
Yes, sorry I'm an idiot :)	NON-TOXIC
wt*? why aren't we just using std::string instead of char*,size_t?	TOXIC
don't do this. you spawned the processes which means you have handles to the processes. kill them directly.	NON-TOXIC

C Training details

All encoders are fine-tuned on a Kaggle T4 GPU with the same recipe: maximum length 256 tokens, an effective batch of 32, up to 4 epochs with the best epoch chosen by validation F_1, AdamW with weight

decay 0.01, a linear schedule with 10% warmup, and fp16. The validation split is a stratified 10% of the training folds. The learning rate is chosen once per model on fold 0's validation split from the grid in Table 7 and then used for all five folds. The TF-IDF baseline keeps word unigrams and bigrams that occur in at least two training comments (minimum document frequency 2). The TF-IDF vocabulary, fitted on the training folds, has 21,906 to 22,301 terms on the clean folds and 54,280 to 55,783 on the leaky folds, and the regularization strength C chosen by inner cross-validation was 4 or 8 in every fold. The minimum document frequency is a likely reason for the baseline's gain on the leaky folds: a word pair that occurs in only one comment is kept when that comment has a copy in the training folds.

Table 7: Encoder runs: validation F_1 on fold 0 per learning rate, chosen learning rate, best epoch per fold, batch and run time.

Model	LR grid	Val. F1 per LR (fold 0)	Chosen	Best epoch per fold	Batch	Min/run
BERT-base	2e-5, 3e-5, 5e-5	0.840 / 0.836 / 0.838	2e-5	4, 3, 3, 4, 2	32×1	4.0
RoBERTa-base	2e-5, 3e-5, 5e-5	0.828 / 0.827 / 0.838	5e-5	3, 3, 3, 2, 4	32×1	4.6
CodeBERT-base	2e-5, 3e-5, 5e-5	0.809 / 0.816 / 0.828	5e-5	4, 4, 3, 2, 3	32×1	4.3
DeBERTa-v3-base	1e-5, 2e-5, 3e-5	0.821 / 0.832 / 0.842	3e-5	3, 3, 4, 3, 3	32×1	7.3
DeBERTa-v3-large	5e-6, 1e-5, 2e-5	0.839 / 0.837 / 0.849	2e-5	4, 4, 4, 3, 3	8×4	18.1
ModernBERT-base	2e-5, 3e-5, 5e-5	0.828 / 0.821 / 0.811	2e-5	3, 4, 4, 2, 3	32×1	7.0
ModernBERT-large	1e-5, 2e-5, 3e-5	0.832 / 0.852 / 0.844	2e-5	3, 4, 3, 3, 4	16×2	16.1
Ettin-150m	2e-5, 3e-5, 5e-5	0.822 / 0.821 / 0.813	2e-5	3, 4, 4, 4, 3	32×1	7.1
Ettin-400m	1e-5, 2e-5, 3e-5	0.834 / 0.835 / 0.837	3e-5	3, 4, 2, 4, 2	16×2	15.1
NeoBERT	2e-5, 3e-5, 5e-5	0.833 / 0.839 / 0.848	5e-5	4, 4, 4, 2, 3	32×1	10.1
BERT-base (leaky)	2e-5	–	2e-5	4, 4, 4, 3, 4	32×1	6.5

D Additional results

The technical words for the jargon slice are the software engineering terms that the ToxiCR authors found toxicity detectors to misread: kill, execute, die, dead, garbage, dummy, junk, dirty, trash, daemon, naked and dump (Sarker et al., 2020, Sec. IV-B), plus bug (Sarker et al., 2023, Sec. 5.6.2). The list is only used for Table 8, which compares each system's false-positive rate on non-toxic comments with such a word against its rate on all non-toxic comments.

Table 8: False-positive rate on non-toxic comments with a technical word, against all non-toxic comments.

System	All, clean (10,271)	Jargon, clean (1,561)	All, subset (1,615)	Jargon, subset (237)
TF-IDF + LR	0.065	0.037	0.064	0.068
Detoxify original	0.040	0.144	0.032	0.097
Detoxify original, cal.	0.151	0.387	0.140	0.338
Detoxify unbiased	0.135	0.299	0.142	0.274
Detoxify unbiased, cal.	0.188	0.409	0.189	0.380
BERT-base	0.039	0.019	0.041	0.021
RoBERTa-base	0.037	0.021	0.040	0.030
CodeBERT-base	0.042	0.021	0.046	0.025
DeBERTa-v3-base	0.038	0.017	0.044	0.017
DeBERTa-v3-large	0.039	0.024	0.044	0.021
ModernBERT-base	0.037	0.020	0.033	0.013
ModernBERT-large	0.032	0.017	0.033	0.013
Ettin-150m	0.040	0.026	0.041	0.030
Ettin-400m	0.033	0.015	0.040	0.017
NeoBERT	0.033	0.015	0.037	0.017
gpt-oss-120b A	–	–	0.054	0.059
gpt-oss-120b B	–	–	0.045	0.042
gpt-oss-120b C	–	–	0.071	0.076
Kimi-K2.6 A	–	–	0.010	0.017
Kimi-K2.6 B	–	–	0.022	0.021
Kimi-K2.6 C	–	–	0.059	0.051

Table 9: All systems on the 2,000-comment subset.

System	Year	Size	Precision	Recall	F1	Pooled F1 [95% CI]	AP
TF-IDF + LR	–	–	0.750 ± 0.043	0.804 ± 0.056	0.776 ± 0.043	0.777 [0.744, 0.806]	0.851
Detoxify original, cal.	2020	110M	0.542 ± 0.052	0.693 ± 0.032	0.608 ± 0.043	0.608 [0.569, 0.646]	0.626
Detoxify unbiased, cal.	2021	125M	0.470 ± 0.059	0.694 ± 0.070	0.557 ± 0.037	0.557 [0.519, 0.595]	0.552
Detoxify unbiased	2021	125M	0.500 ± 0.066	0.595 ± 0.061	0.542 ± 0.057	0.544 [0.503, 0.583]	0.552
Detoxify original	2020	110M	0.726 ± 0.031	0.350 ± 0.064	0.469 ± 0.061	0.470 [0.419, 0.520]	0.626
BERT-base	2018	110M	0.835 ± 0.040	0.874 ± 0.039	0.854 ± 0.032	0.854 [0.827, 0.879]	0.915
DeBERTa-v3-base	2021	184M	0.823 ± 0.035	0.852 ± 0.027	0.836 ± 0.016	0.837 [0.807, 0.864]	0.920
DeBERTa-v3-large	2021	435M	0.823 ± 0.035	0.849 ± 0.027	0.835 ± 0.016	0.835 [0.806, 0.863]	0.917
RoBERTa-base	2019	125M	0.833 ± 0.023	0.831 ± 0.050	0.832 ± 0.030	0.832 [0.802, 0.860]	0.909
CodeBERT-base	2020	125M	0.812 ± 0.045	0.824 ± 0.044	0.817 ± 0.033	0.817 [0.787, 0.845]	0.903
NeoBERT	2025	250M	0.848 ± 0.055	0.840 ± 0.073	0.841 ± 0.021	0.841 [0.814, 0.868]	0.911
ModernBERT-large	2024	395M	0.854 ± 0.018	0.812 ± 0.049	0.832 ± 0.030	0.834 [0.803, 0.862]	0.916
Ettin-400m	2025	400M	0.834 ± 0.067	0.831 ± 0.060	0.830 ± 0.032	0.830 [0.799, 0.856]	0.892
Ettin-150m	2025	150M	0.827 ± 0.053	0.819 ± 0.057	0.821 ± 0.035	0.821 [0.791, 0.850]	0.901
ModernBERT-base	2024	149M	0.850 ± 0.052	0.792 ± 0.068	0.818 ± 0.044	0.818 [0.787, 0.847]	0.904
gpt-oss-120b C	2025	117B	0.720 ± 0.031	0.755 ± 0.059	0.735 ± 0.020	0.735 [0.700, 0.770]	–
Kimi-K2.6 C	2026	$\sim$1T	0.744 ± 0.052	0.704 ± 0.034	0.722 ± 0.025	0.722 [0.682, 0.758]	–
gpt-oss-120b B	2025	117B	0.767 ± 0.054	0.609 ± 0.036	0.678 ± 0.026	0.677 [0.636, 0.716]	–
Kimi-K2.6 B	2026	$\sim$1T	0.831 ± 0.039	0.466 ± 0.050	0.596 ± 0.047	0.599 [0.552, 0.646]	–
gpt-oss-120b A	2025	117B	0.540 ± 0.089	0.266 ± 0.048	0.356 ± 0.061	0.358 [0.308, 0.406]	–
Kimi-K2.6 A	2026	$\sim$1T	0.644 ± 0.089	0.080 ± 0.026	0.141 ± 0.044	0.144 [0.100, 0.191]	–

Table 10: Paired bootstrap comparisons (difference in pooled F_1 with 95% interval), not adjusted for multiple comparisons. * = an interval end is within 0.002 of 0.

RQ	A − B	Comments	F1 A / B	Difference	95% CI	Excl. 0
RQ1	BERT-base − TF-IDF + LR	12,722 clean	0.834 / 0.761	+0.072	[+0.060, +0.085]	yes
RQ1	BERT-base − TF-IDF + LR	19,647 leaky	0.893 / 0.860	+0.032	[+0.025, +0.040]	yes
RQ2	BERT-base − Detoxify original, cal.	12,722 clean	0.834 / 0.581	+0.253	[+0.236, +0.270]	yes
RQ2	RoBERTa-base − Detoxify unbiased, cal.	12,722 clean	0.837 / 0.562	+0.275	[+0.258, +0.290]	yes
RQ2	TF-IDF + LR − Detoxify original, cal.	12,722 clean	0.761 / 0.581	+0.180	[+0.163, +0.198]	yes
RQ3	DeBERTa-v3-base − TF-IDF + LR	12,722 clean	0.844 / 0.761	+0.083	[+0.071, +0.095]	yes
RQ3	RoBERTa-base − CodeBERT-base	12,722 clean	0.837 / 0.825	+0.012	[+0.004, +0.021]	yes
RQ3	DeBERTa-v3-base − NeoBERT	12,722 clean	0.844 / 0.831	+0.013	[+0.003, +0.022]	yes
RQ3	DeBERTa-v3-base − ModernBERT-base	12,722 clean	0.844 / 0.826	+0.018	[+0.009, +0.028]	yes
RQ3	DeBERTa-v3-base − Ettin-150m	12,722 clean	0.844 / 0.818	+0.026	[+0.015, +0.036]	yes
RQ3	ModernBERT-large − ModernBERT-base	12,722 clean	0.830 / 0.826	+0.004	[−0.005, +0.013]	no
RQ3	Ettin-400m − Ettin-150m	12,722 clean	0.835 / 0.818	+0.017	[+0.006, +0.027]	yes
RQ3	DeBERTa-v3-large − DeBERTa-v3-base	12,722 clean	0.841 / 0.844	−0.003	[−0.011, +0.005]	no
RQ3	DeBERTa-v3-base − Ettin-400m	12,722 clean	0.844 / 0.835	+0.009	[−0.000, +0.019]	no*
RQ4	gpt-oss-120b B − gpt-oss-120b A	2,000 subset	0.677 / 0.358	+0.319	[+0.268, +0.371]	yes
RQ4	gpt-oss-120b C − gpt-oss-120b B	2,000 subset	0.735 / 0.677	+0.058	[+0.026, +0.089]	yes
RQ4	Kimi-K2.6 B − Kimi-K2.6 A	2,000 subset	0.599 / 0.144	+0.455	[+0.399, +0.510]	yes
RQ4	Kimi-K2.6 C − Kimi-K2.6 B	2,000 subset	0.722 / 0.599	+0.123	[+0.084, +0.161]	yes
RQ4	gpt-oss-120b A − Kimi-K2.6 A	2,000 subset	0.358 / 0.144	+0.215	[+0.165, +0.262]	yes
RQ4	gpt-oss-120b B − Kimi-K2.6 B	2,000 subset	0.677 / 0.599	+0.078	[+0.039, +0.118]	yes
RQ4	gpt-oss-120b C − Kimi-K2.6 C	2,000 subset	0.735 / 0.722	+0.013	[−0.021, +0.049]	no
RQ4	DeBERTa-v3-base − gpt-oss-120b C	2,000 subset	0.837 / 0.735	+0.102	[+0.066, +0.139]	yes
RQ4	TF-IDF + LR − gpt-oss-120b C	2,000 subset	0.777 / 0.735	+0.042	[+0.000, +0.082]	yes*

E Use of generative AI

I used Claude Code throughout the project to help write code and troubleshoot problems as they came up. Claude was also used to help with phrasing in the written text. I reviewed and edited all suggestions before including them in the report.